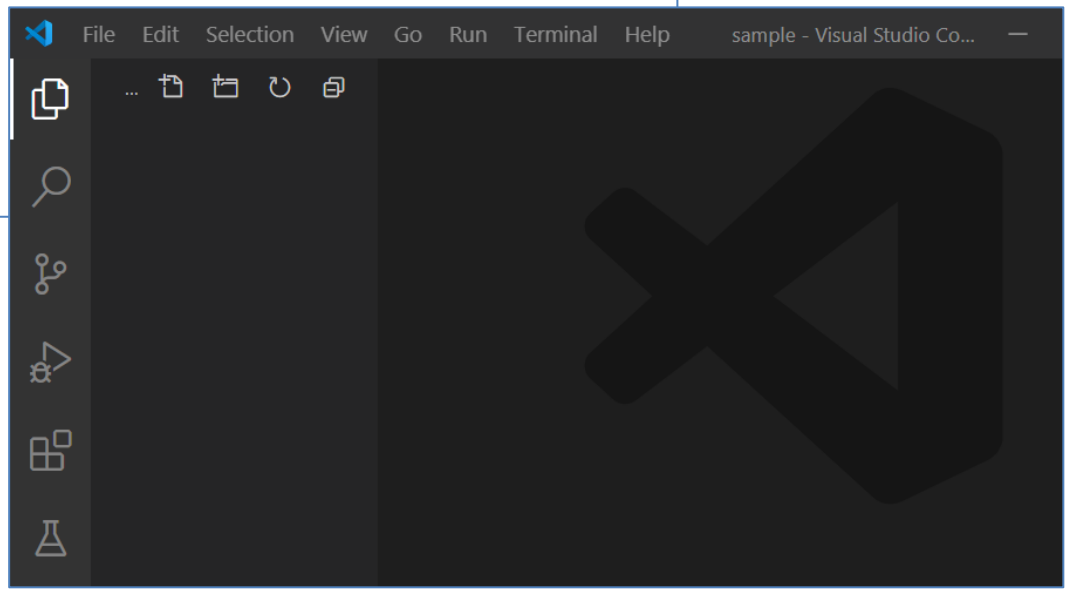
■ Flask 설치

● pip install flask

```
(work_flask) C:₩work_flask>pip install flask
Collecting flask
  Downloading https://files.pythonhosted.org/packages/9b/93/628509b8d5dc749656a96
    |████████████████████████████████| 102kB
Collecting click>=5.1
  Downloading https://files.pythonhosted.org/packages/fa/37/45185cb5abbc30d7257104
    |████████████████████████████████| 81kB 1
Collecting Werkzeug>=0.15
  Using cached https://files.pythonhosted.org/packages/ce/42/3aeda98f96e85fd2618053
Collecting Jinja2>=2.10.1
  Using cached https://files.pythonhosted.org/packages/65/e0/eb35e762802015cab1ccee
Collecting itsdangerous>=0.24
  Downloading https://files.pythonhosted.org/packages/76/ae/44b03b253d6fade317f32c
Collecting MarkupSafe>=0.23
  Downloading https://files.pythonhosted.org/packages/b9/82/833c7714951bff8f502ed0
Installing collected packages: click, Werkzeug, MarkupSafe, Jinja2, itsdangerous, flask
Successfully installed Jinja2-2.10.3 MarkupSafe-1.1.1 Werkzeug-0.16.0 click-7.0 flask-1.1.

(work_flask) C:₩work_flask>
```

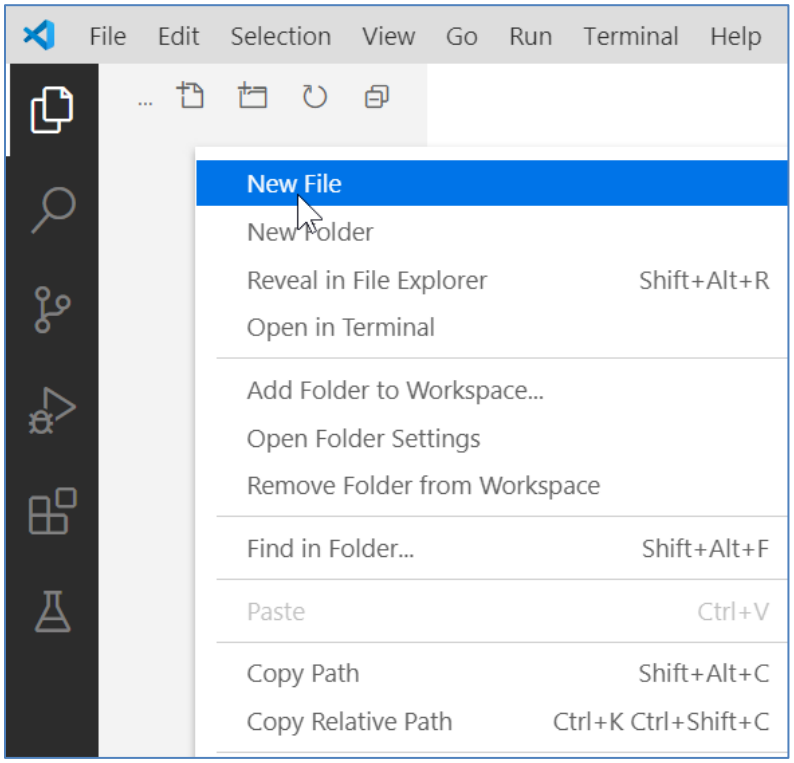
■ 프로젝트 생성

```
C:₩Windows₩system32₩cmd.exe
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:₩Users₩GGoReb>cd /

C:₩>mkdir flask

C:₩>cd flask

C:₩flask>mkdir sample

C:₩flask>cd sample

C:₩flask₩sample>code .
```

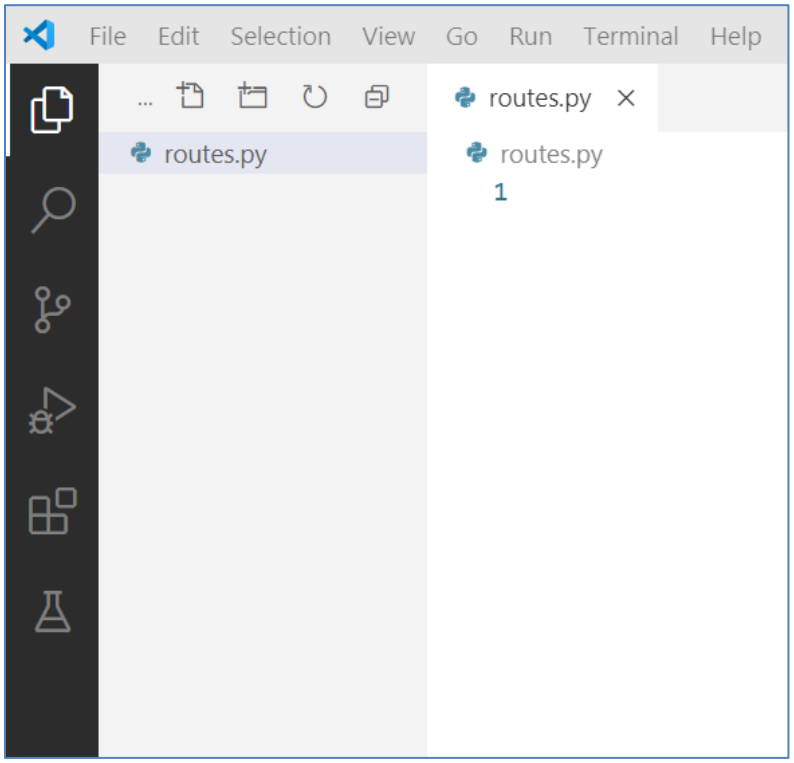


■ 파일 생성

● 마우스 우클릭 → New File



● routes.py

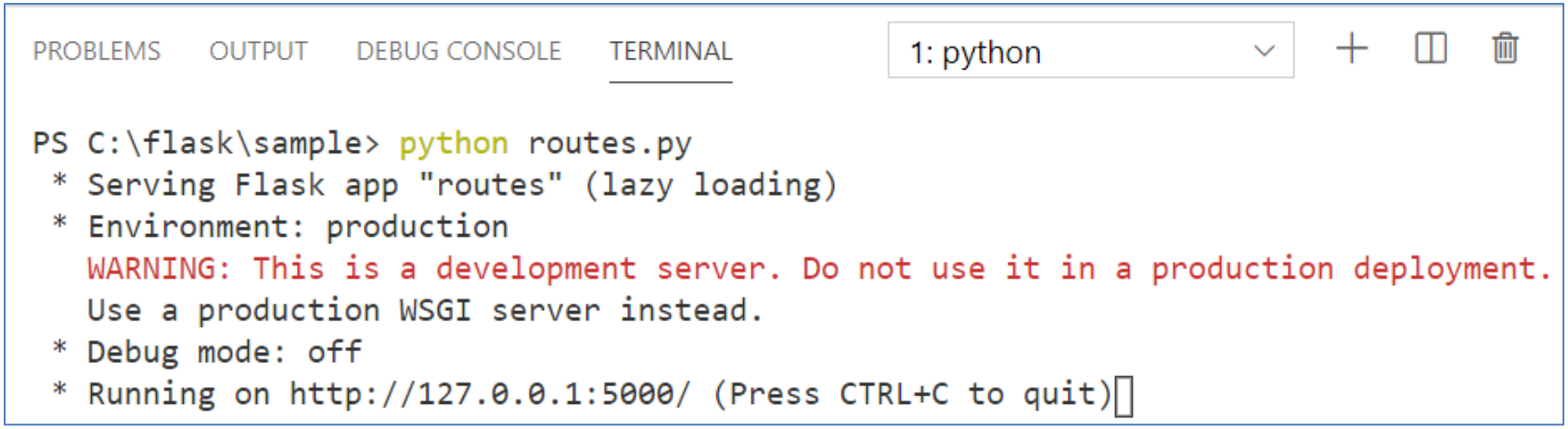


■ Flask 실행

● 기본 구성

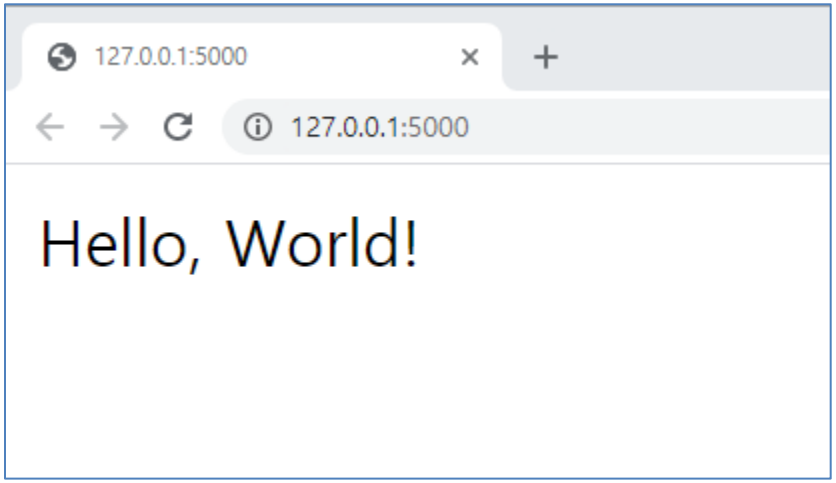
```python
from flask import Flask  # import flask module
app = Flask(__name__)  # 초기화

@app.route('/')  # 요청 주소
def hello_world():  # 함수
  return 'Hello, World!'  # 응답 결과

if __name__ == '__main__':
  app.run()  # flask 실행
```

● routes.py 실행 : Ctrl + J (TERMINAL) → python routes.py



■ Flask 실행

● 실행 결과 : http://127.0.0.1:5000 (http://localhost:5000)



● 공식 문서 : https://flask.palletsprojects.com/

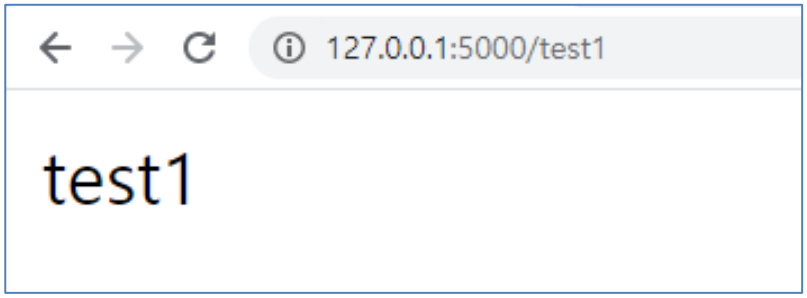
● 코드 수정 & 저장 시 자동 재실행

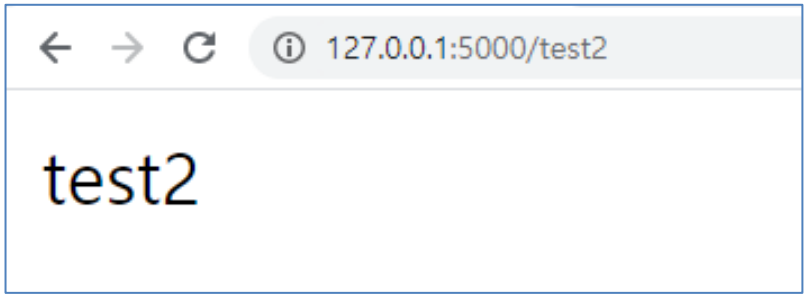
```
…

if __name__ == '__main__':
    app.run(debug=True)
```

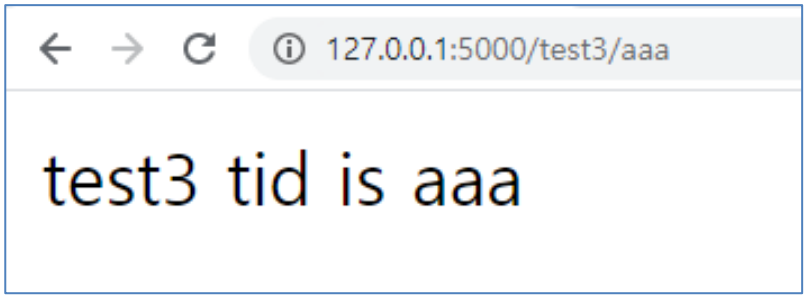
■ Routing

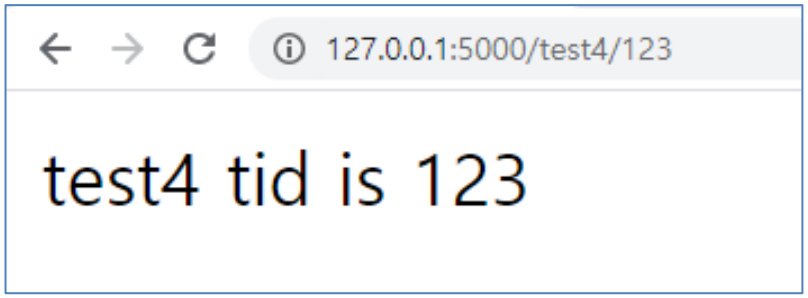
```
@app.route('/test1')
def test1():
  return 'test1'

@app.route('/test2', methods=[ 'GET', 'POST' ])
def test2():
  return 'test2'

@app.route('/test3/<tid>')
def test3(tid):
  return 'tid is %s' % tid

@app.route('/test4')
@app.route('/test4/<int:tid>')
def test4(tid=None):
  return 'test4 tid is %s' % tid
```

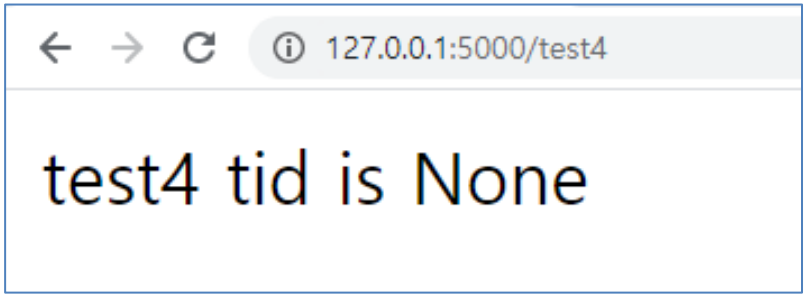
■ Routing


127.0.0.1:5000/test1
test1


127.0.0.1:5000/test2
test2


127.0.0.1:5000/test3/aaa
test3 tid is aaa


127.0.0.1:5000/test4/123
test4 tid is 123


127.0.0.1:5000/test4
test4 tid is None

■ Template

- 특정 문자열을 반환하는 것으로 웹 페이지를 보여주는 것도 가능하지만 사진, 영상 등의 정적 파일을 보여주거나 CSS/JS를 적용하기 위해서는 별도로 작성된 HTML(Template)을 사용하는 것을 권장

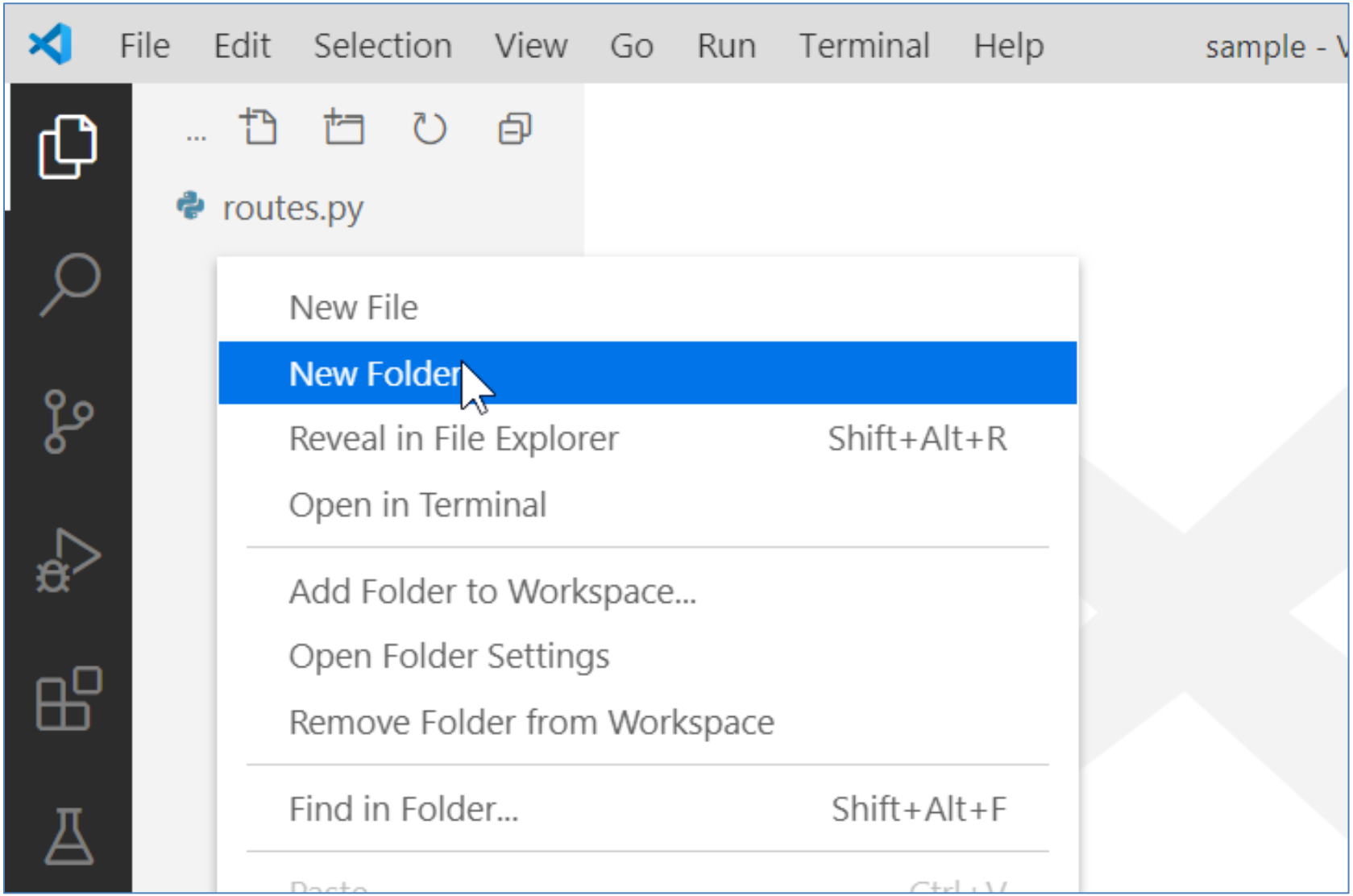
- Flask에서는 Jinja2 라는 이름의 Template Engine을 사용

- HTML 파일을 보관하기 위한 폴더를 따로 만들어주어야 하며 반드시 플라스크를 실행시키는 코드와 같은 경로에서 templates 라는 이름으로 구성되어야 함

- Jinja2 공식 사이트 - https://jinja.palletsprojects.com/

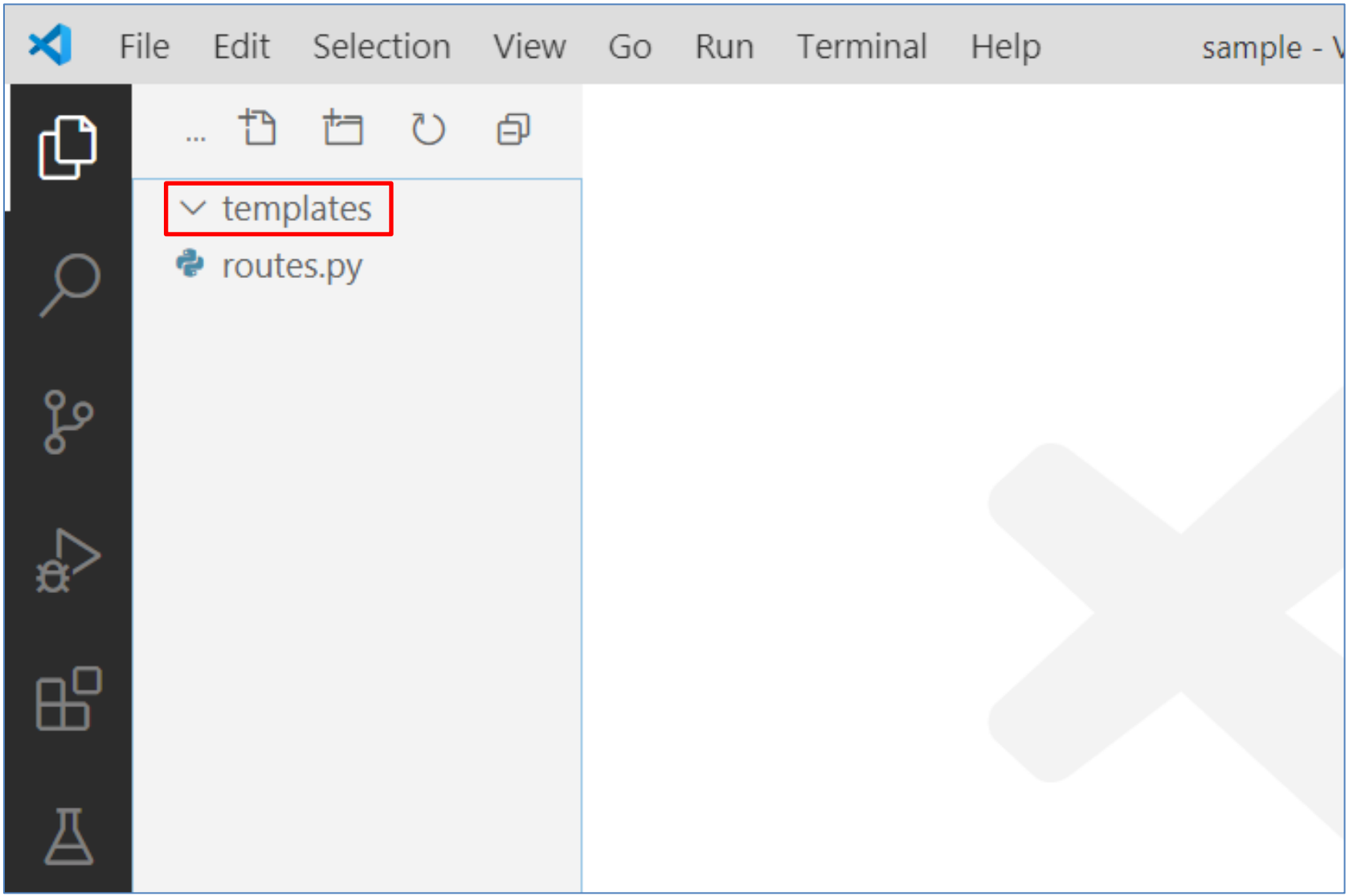
■ Template

● templates 폴더 생성



■ Template

● templates 폴더 생성



■ Template

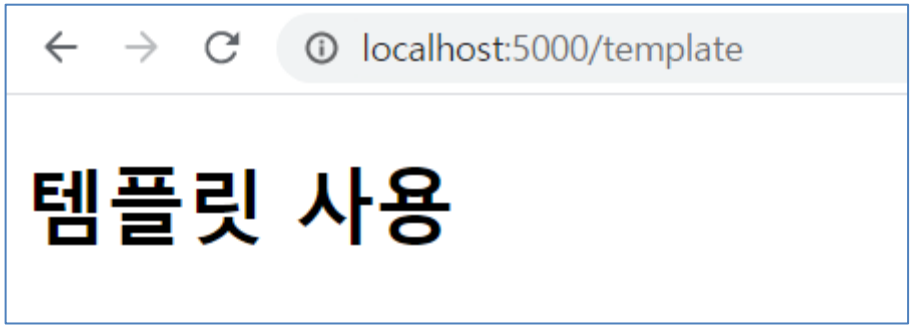
● templates/template.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Document</title>
</head>
<body>
  <h1>템플릿 사용</h1>
</body>
</html>
```

■ Template

● routes.py

```
…
from flask import render_template
…

@app.route('/template')
def template():
  return render_template('template.html')


…
```



■ Template (Context Param 사용)

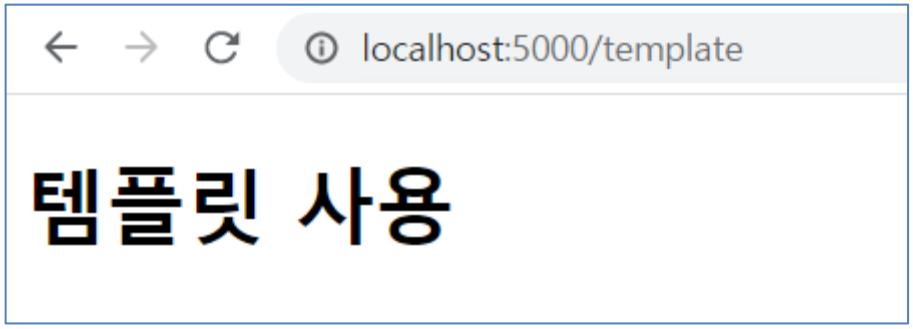
● templates/template_with_param.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Document</title>
</head>
<body>
  <h1>템플릿 사용</h1>
  <h1>name - {{ name }}</h1>
</body>
</html>
```

■ Template (Context Param 사용)

● routes.py

```
…
from flask import render_template
…

@app.route('/template_with_param')
def template_with_param():
  user = 'GGoReb'
  return render_template(
    'template_with_param.html', name=user)

…
```

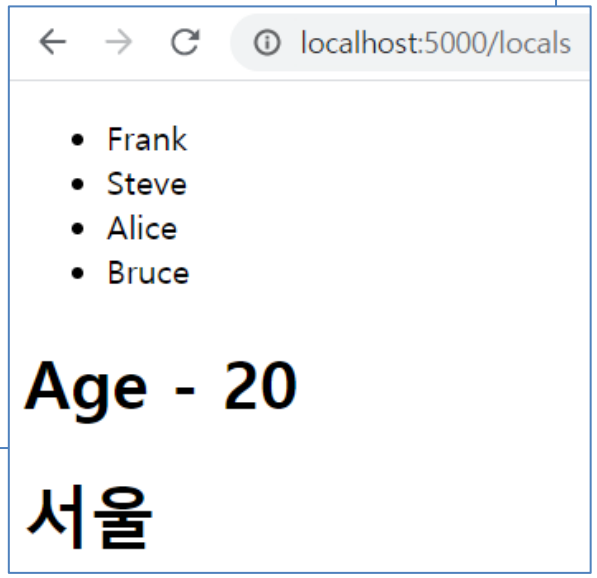


■ Template (locals)

● 해당 함수 내의 모든 변수를 Context Param으로 사용

```python
@app.route("/locals")
def local():
  users = ['Frank', 'Steve', 'Alice', 'Bruce']
  age = 20
  address_dict = {'a':'서울', 'b':'대전', 'c':'제주'}
  return render_template('locals.html', **locals())
```

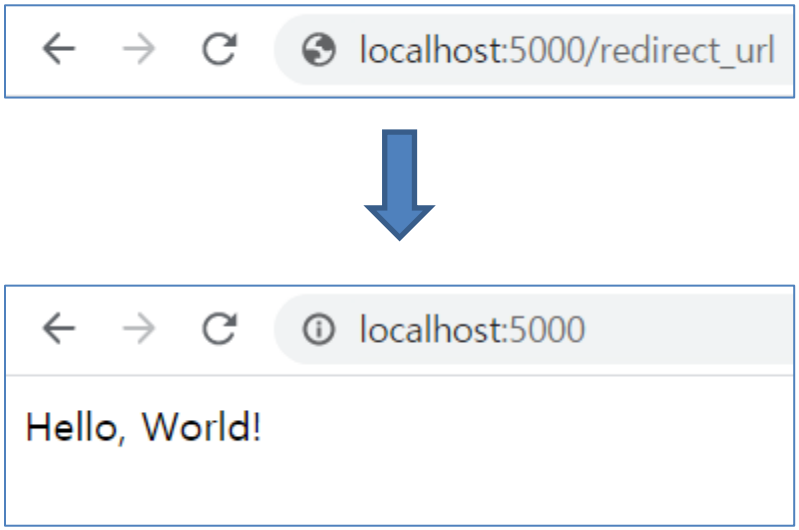
```
  <ul>
    {% for user in users %}
      <li>{{ user }}</li>
    {% endfor %}
  </ul>
  {% if age %}
    <h1>Age - {{ age }}</h1>
  {% endif %}
  <h1>{{ address_dict['a'] }}</h1>
```



■ Redirect

- 현재의 사용자 요청은 무시하고 새로운 요청 실행

```
from flask import redirect

…

@app.route("/redirect_url")
def redirect_url():
  return redirect('/')
```



■ Redirect

- redirect('url') 또는 redirect(url_for('함수명')) 사용 가능

```
@app.route('/')
def hello_world():
  return 'Hello, World!'

from flask import url_for

@app.route("/redirect_url")
def redirect_url():
  return redirect('/')
  # 또는 redirect(url_for('hello_world'))
```

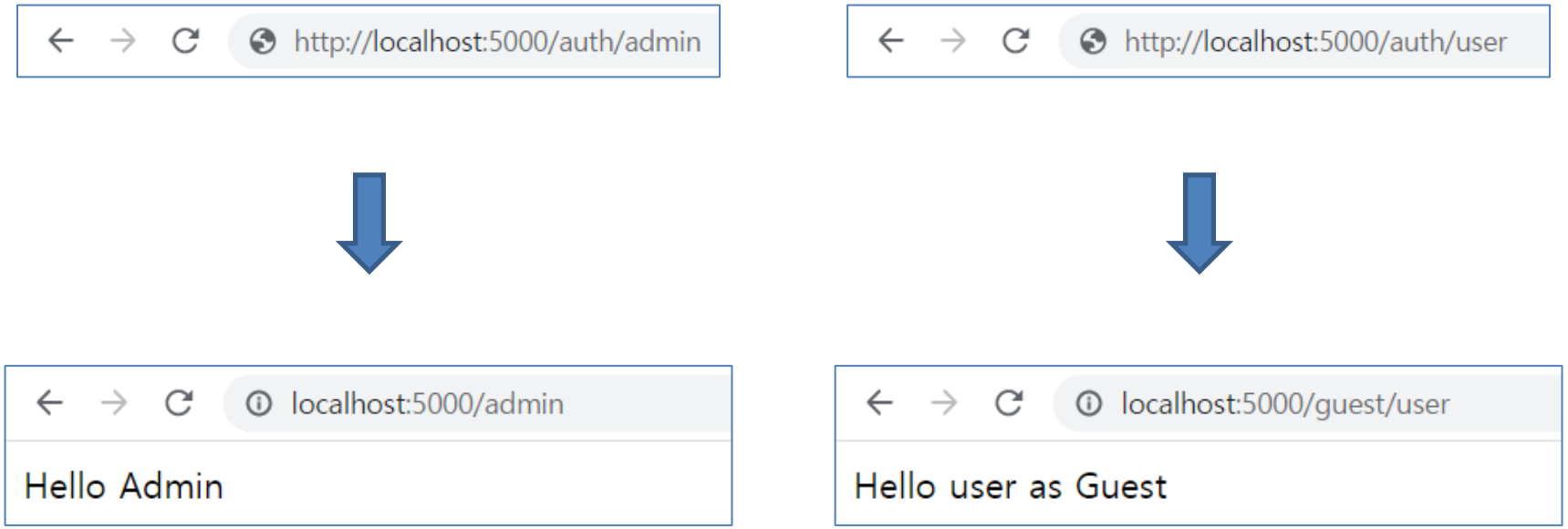
■ Redirect

● /auth/[계정] 접속 후 계정별 redirect 처리

```
@app.route("/auth/<account>")
def auth(account):
  if account == 'admin':
    return redirect('/admin')
  else:
    return redirect(url_for('guest', account=account))

@app.route("/admin")
def admin():
  return 'Hello Admin'

@app.route("/guest/<account>")
def guest(account):
  return 'Hello %s as Guest' % account
```

■ Redirect

- /auth/[계정] 접속 후 계정별 redirect 처리

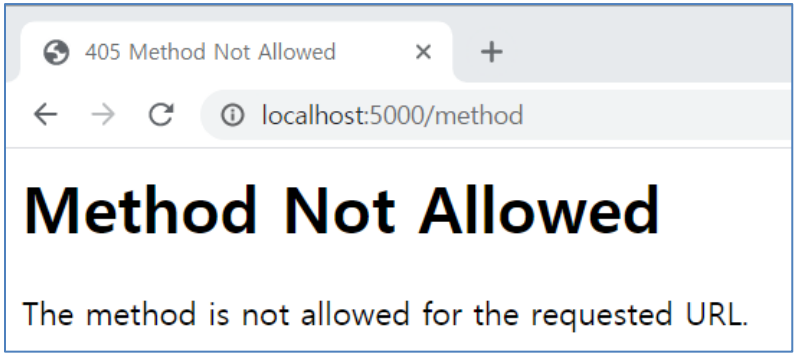


■ HTTP Method

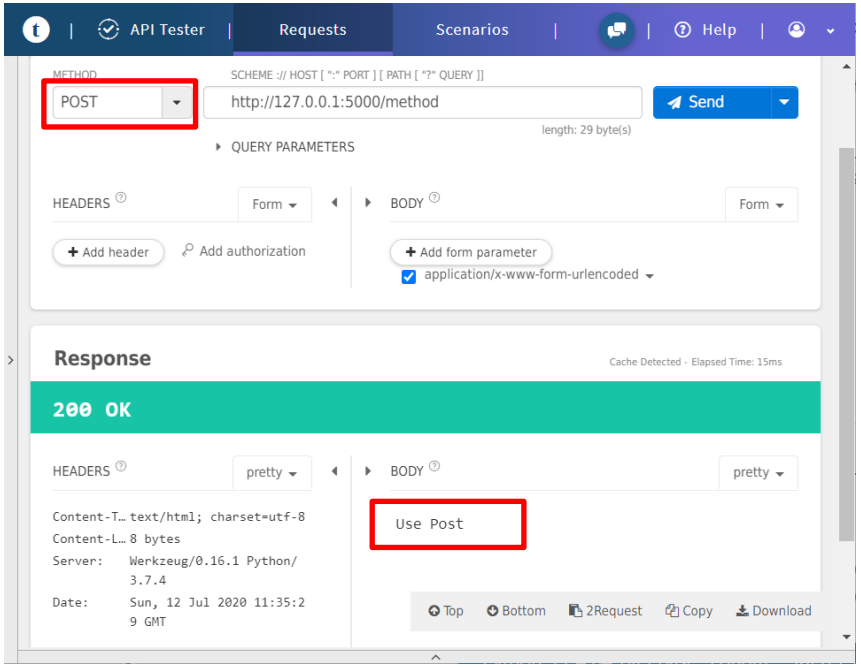
● POST 메소드 사용

```
@app.route("/method", methods=['POST'])
def method():
  return 'Use Post'
```

Browser (Get)



Rest Client



■ HTTP Method

● GET / POST 구분

```
from flask import request

@app.route("/join", methods=['GET', 'POST'])
def join():
  if request.method == 'GET':
    return render_template('join.html')
  else :
    # 데이터베이스 작업
    return '가입완료'
```

- GET ➔ join.html (회원가입 양식)
- POST ➔ 회원가입 로직 수행

■ HTTP Method

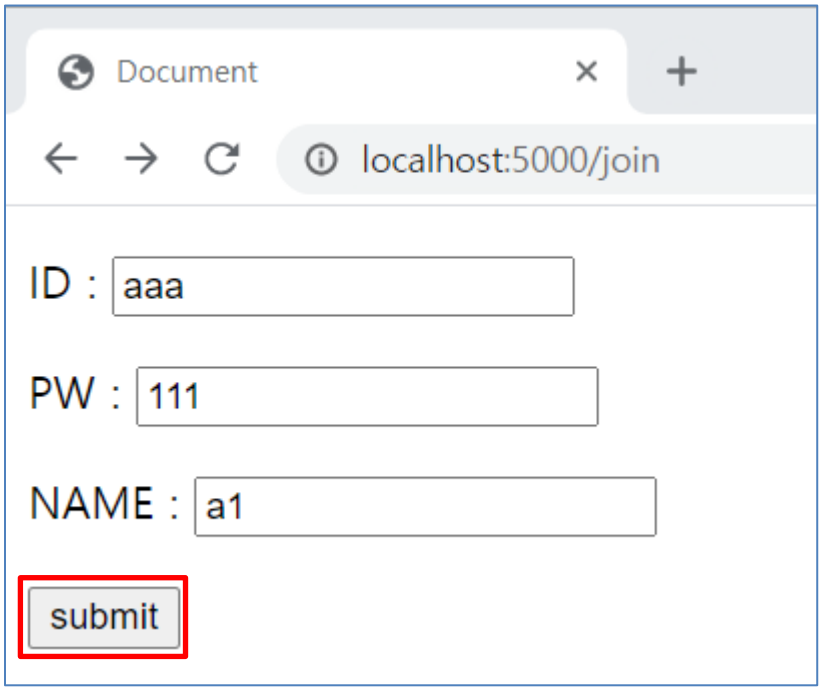
● join.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Document</title>
</head>
<body>
  <form action="/join" method="POST">
    <p>ID : <input type="text" name='user_id'></p>
    <p>PW : <input type="text" name='user_pw'></p>
    <p>NAME : <input type="text" name='user_name'></p>
    <p><input type="submit" value='submit'></p>
  </form>
</body>
</html>
```

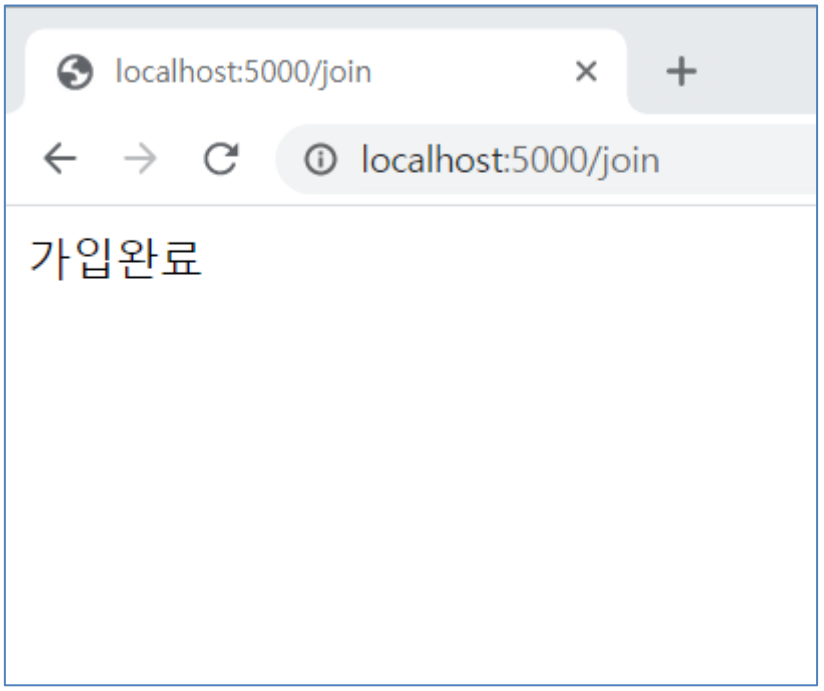
■ HTTP Method

● 실행 결과

[GET]



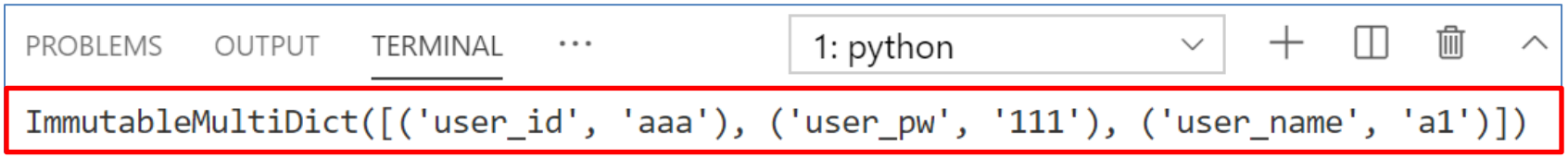
[POST]



■ HTTP Method

● FORM 데이터 활용

```python
from flask import request

@app.route("/join", methods=['GET', 'POST'])
def join():
  if request.method == 'GET':
    return render_template('join.html')
  else :
    form = request.form
    print(form)

    # 데이터베이스 작업
    return '가입완료'
```



■ HTTP Method

● FORM 데이터 활용 (POST)

```
form = request.form

user_id = form['user_id']  # form.get('user_id')
user_pw = form['user_pw']  # form.get('user_pw')
user_name = form['user_name']  # form.get('user_name')
```

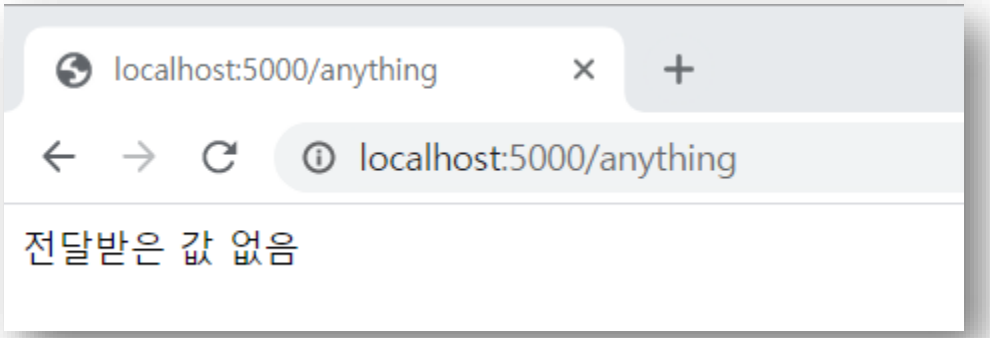
● FORM 데이터 활용 (GET)

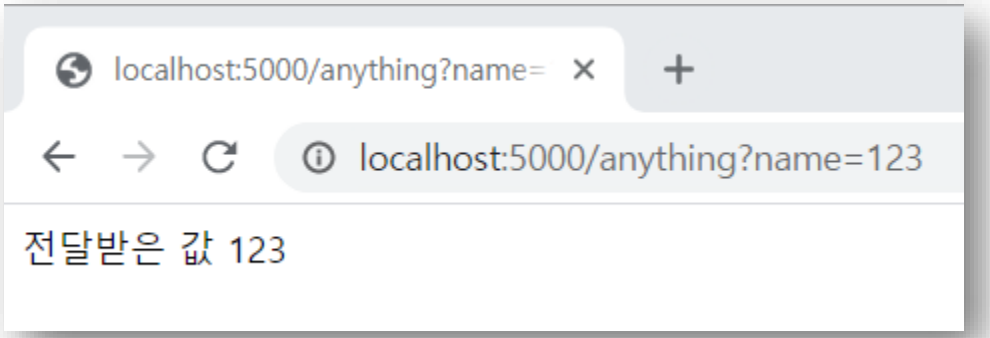
```
form = request.args

user_id = form['user_id']  # form.get('user_id')
user_pw = form['user_pw']  # form.get('user_pw')
user_name = form['user_name']  # form.get('user_name')
```

■ HTTP Method

● GET / POST 구분없이 파라미터 사용

```
@app.route("/anything")
def anything():
  name = request.values.get('name', default="없음")
  return '전달받은 값 %s' % name
```



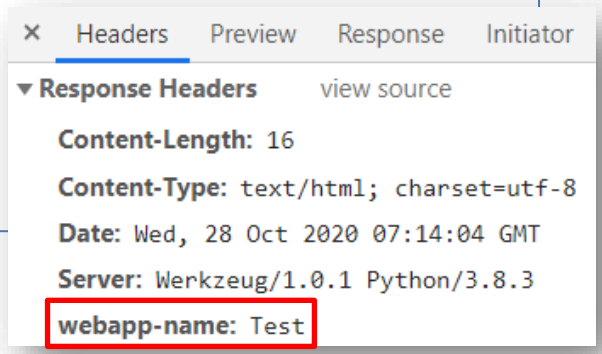


■ 응답처리 Response

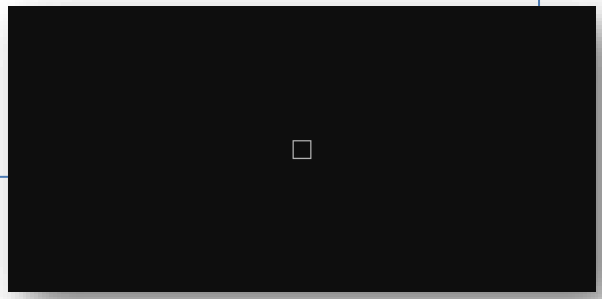
● 자주 사용되는 인자

- status : HTTP 상태 코드 (200 / 404 / 500 등)
- headers : 응답 헤더
- content_type : 응답 형태 (Image, HTML, JSON 등)

```
@app.route("/res")
def res():
  result = Response('응답 테스트')
  result.headers.add('webapp-name', 'Test')
  return result
```

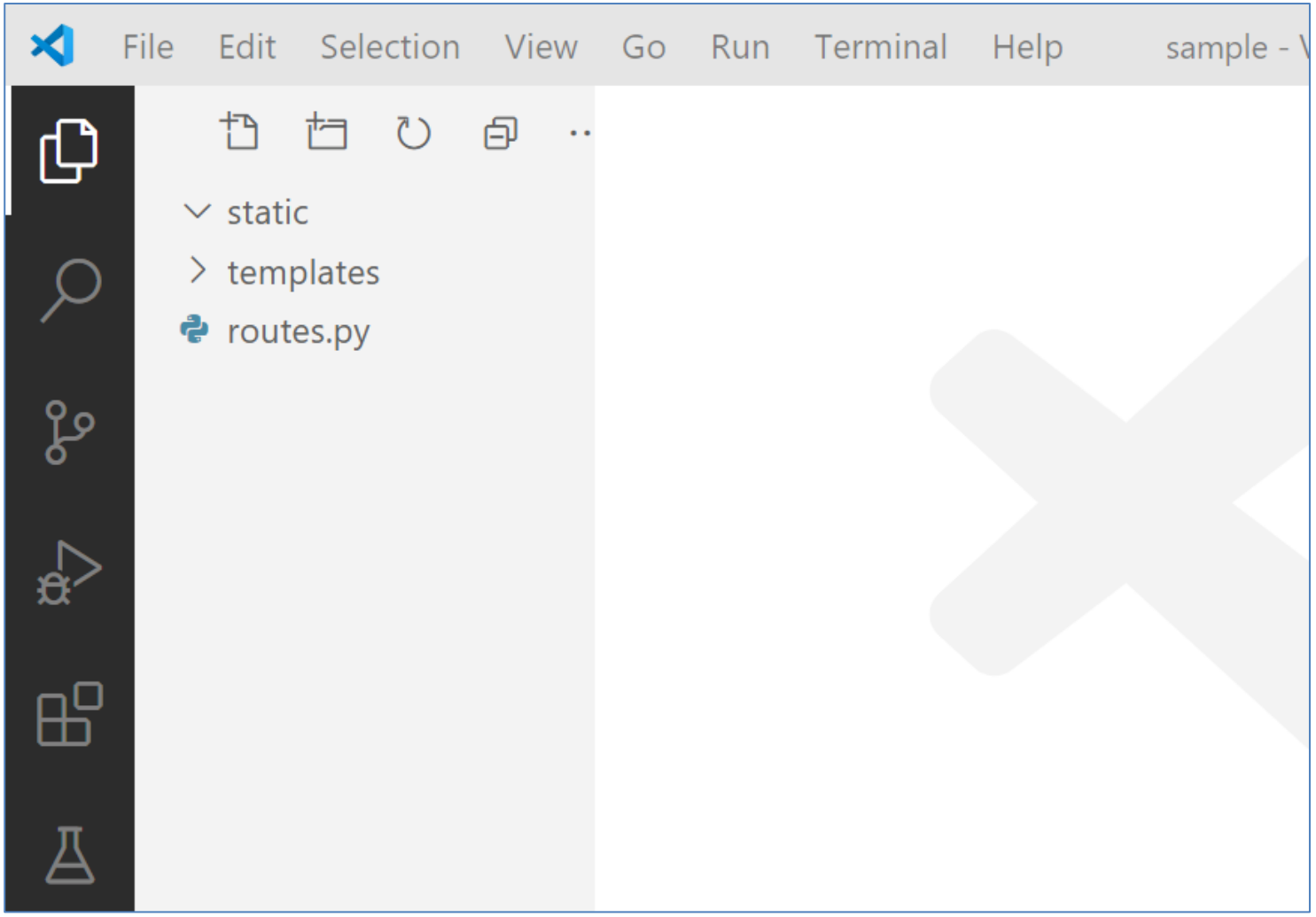



```
@app.route("/res")
def res():
  result = Response('응답 테스트')
  result.content_type = 'image/jpeg'
  return result
```



■ static 파일 적용

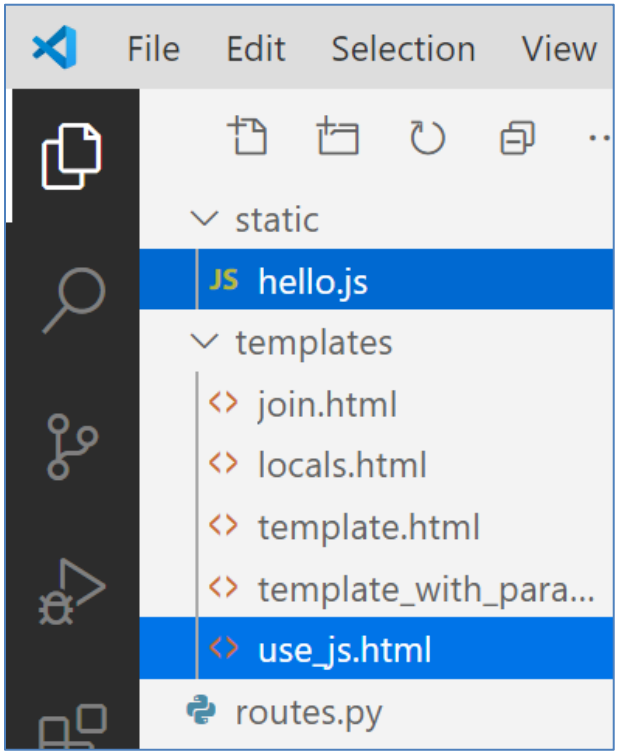
● CSS/JS 등 정적 파일을 보관하기 위한 폴더를 따로 만들어주어야 하며 static 이라는 이름으로 구성되어야 함



■ static 파일 적용

● routes.py

```
@app.route("/use_js")
def use_js():
  return render_template('use_js.html')
```




```
function run() {
  alert('Hello World');
}
```

■ static 파일 적용

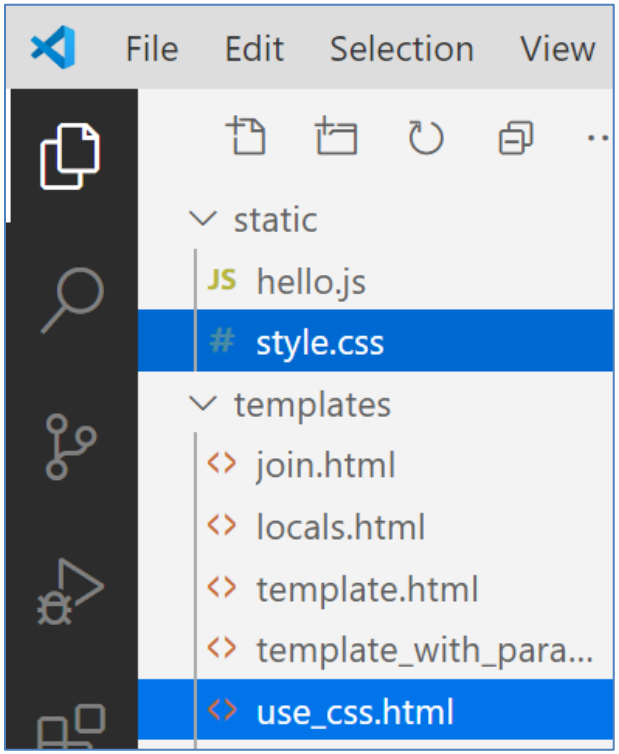
● use_js.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Document</title>
</head>
<body>
  <button type="button" onclick='run()'>실행</button>
  <script
    src="/static/hello.js">
  </script>
</body>
</html>
```

■ static 파일 적용

● routes.py

```
@app.route("/use_css")
def use_css():
  return render_template('use_css.html')
```



■ static 파일 적용

● use_css.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Document</title>
  <link rel="stylesheet" href="/static/style.css">
</head>
<body>
  <form action="/login" method='post'>
    <div class='login'>
      <div class='app-title'>
        <h1>Login</h1>
      </div>
      <div class='login-form'>
        <div class='control-group'>
          <input type="text" class='login-field' placeholder="username" name='username'>
          <label class='login-field-icon fui-user' for="login-name"></label>
        </div>
        <div class='control-group'>
          <input type="password" class='login-field' placeholder="password" name='password'>
          <label class='login-field-icon fui-lock' for="login-pass"></label>
        </div>
        <input type="submit" value='Log in' class='btn btn-primary btn-large btn-block'>
      </div>
    </div>
  </form>
</body>
</html>
```

■ static 파일 적용

● style.css

```css
* {
  box-sizing: border-box;
}

*:focus {
  outline: none;
}

body {
  font-family: Arial;
  background-color: #3498D8;
  padding: 50px;
}

.login {
  margin: 20px auto;
  width: 300px;
}

.login-form {
  text-align: center;
}

.login-screen {
  background-color: #FFF;
  padding: 20px;
  border-radius: 5px;
}
```

■ static 파일 적용

● style.css

```css
.login-link {
  font-size: 12px;
}

.app-title {
  text-align: center;
  color: #777;
}

.control-group {
  margin-bottom: 10px;
}

input {
  text-align: center;
  background-color: #ECF0F1;
  border: 2px solid transparent;
  border-radius: 3px;
  font-size: 16px;
  font-weight: 200;
  padding: 10px 0;
  width: 250px;
  transition: border .5s;
}

input:focus {
  border: 2px solid #3498DB;
  box-shadow: none;
}
```

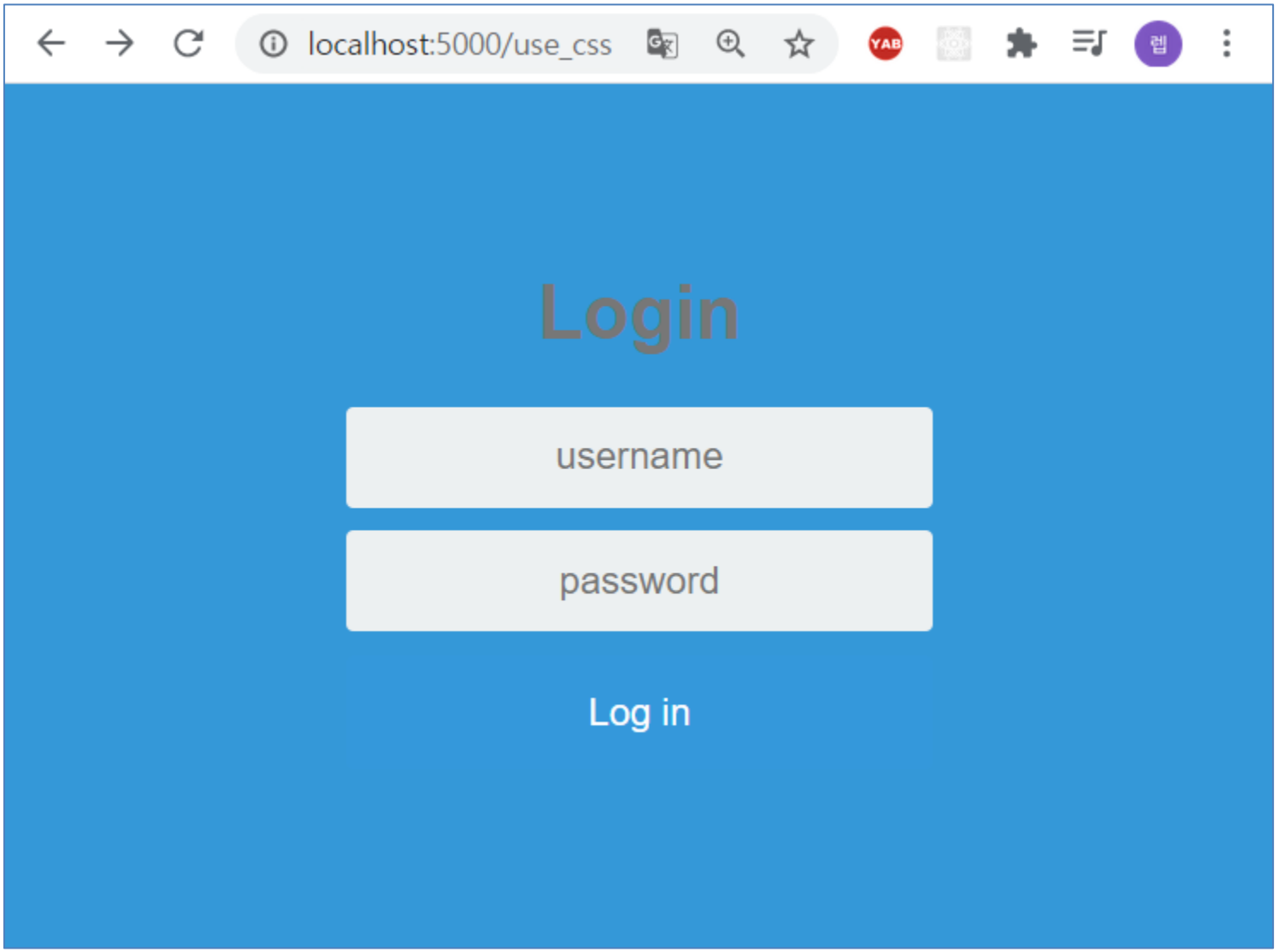
■ static 파일 적용

● style.css

```css
.btn {
  border: 2px solid transparent;
  background: #3498DB;
  color: #ffffff;
  font-size: 16px;
  line-height: 25px;
  padding: 10px 0;
  text-decoration: none;
  text-shadow: none;
  border-radius: 3px;
  box-shadow: none;
  transition: 0.25s;
  display: block;
  width: 250px;
  margin: 0 auto;
}

.btn:hover {
  background-color: #2980B9;
}
```

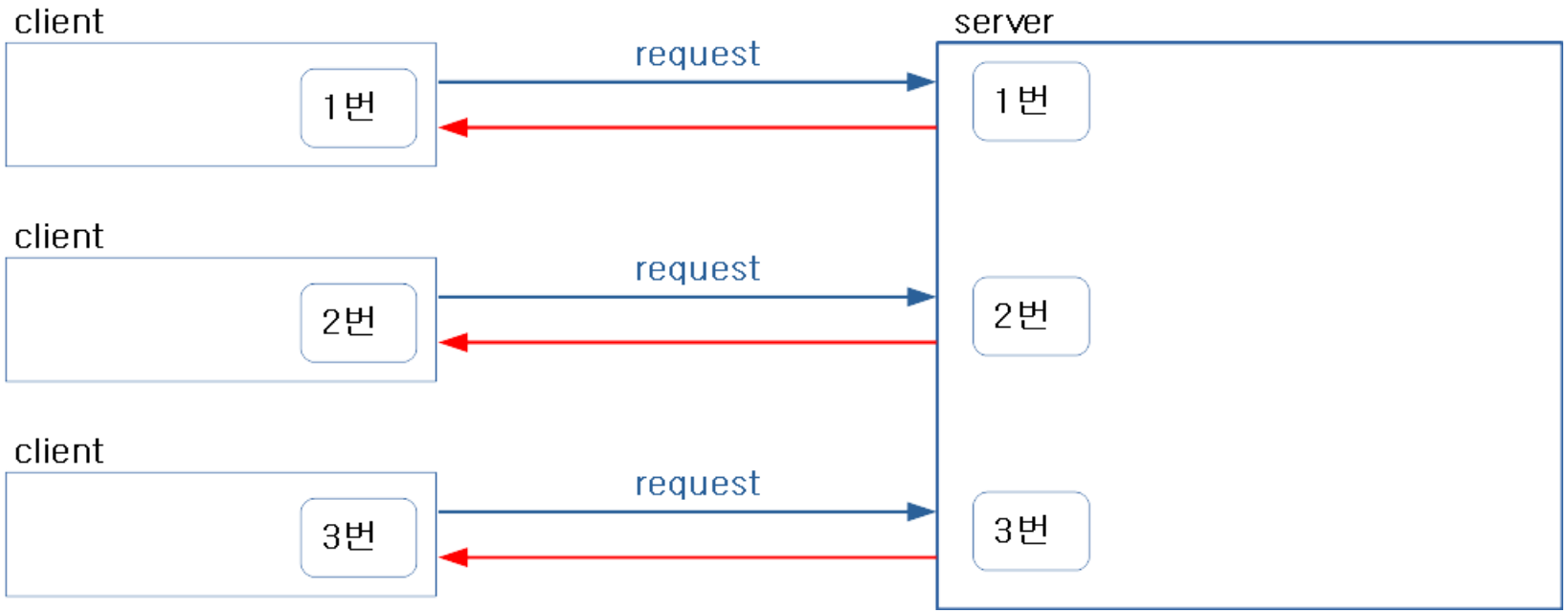
■ static 파일 적용

● 실행 결과



■ cookie

- 쿠키는 클라이언트 브라우저에 텍스트 파일 형태로 저장
- 일반적으로 시간이 지나면 만료기간에 따라 자동으로 삭제
- 자동 로그인, 팝업창에서 "하루동안 창 띄우지 않기" 등의 기능으로 적용



■ cookie

● routes.py (쿠키 저장)

```
from flask import make_response

@app.route("/set_cookie", methods=['get'])
def set_cookie():
  return render_template('set_cookie.html')

@app.route("/set_cookie", methods=['post'])
def set_cookie_post():
  user = request.form['nm']
  resp = make_response('Cookie Setting Complete')
  resp.set_cookie('userID', user)
  return resp
```

■ cookie

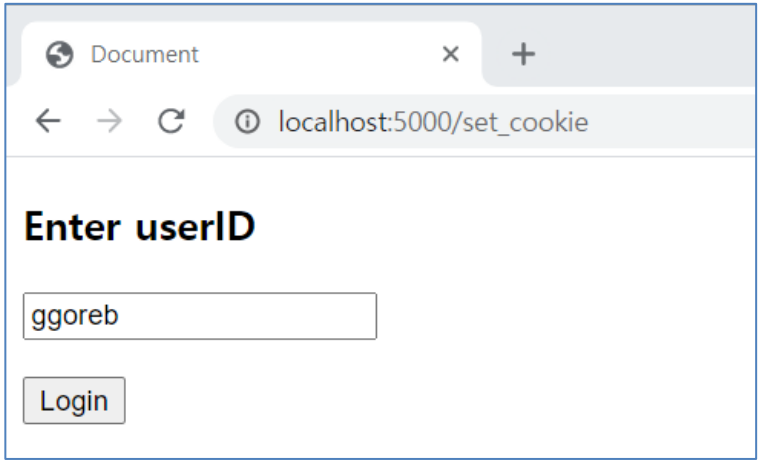
● templates/set_cookie.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Document</title>
</head>
<body>
  <form action="/set_cookie" method='POST'>
    <p><h3>Enter userID</h3></p>
    <p><input type="text" name='nm'></p>
    <p><input type="submit" value='Login'></p>
  </form>
</body>
</html>
```

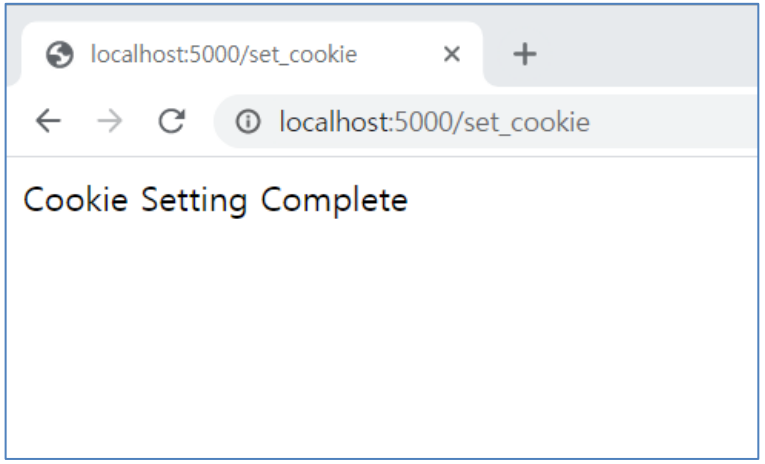
■ cookie

● 실행 결과

[GET]



[POST]

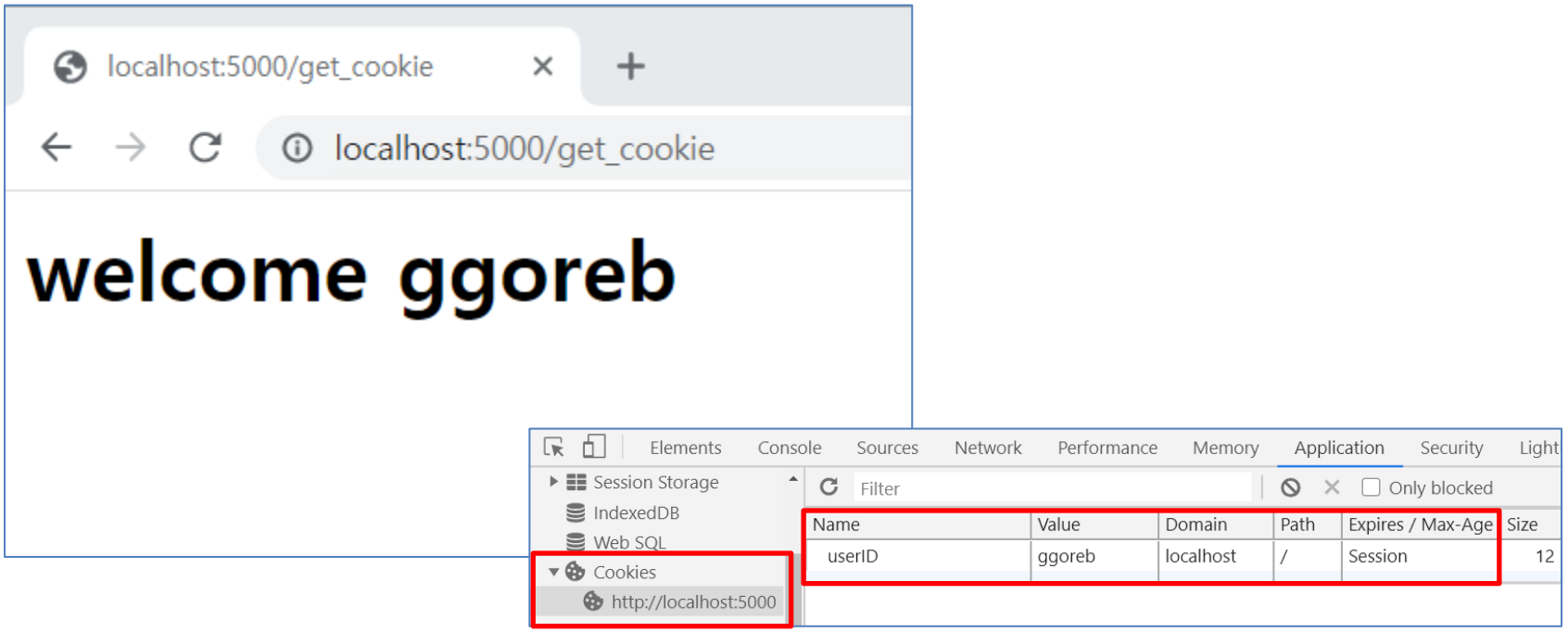


■ cookie

● routes.py (쿠키 조회)

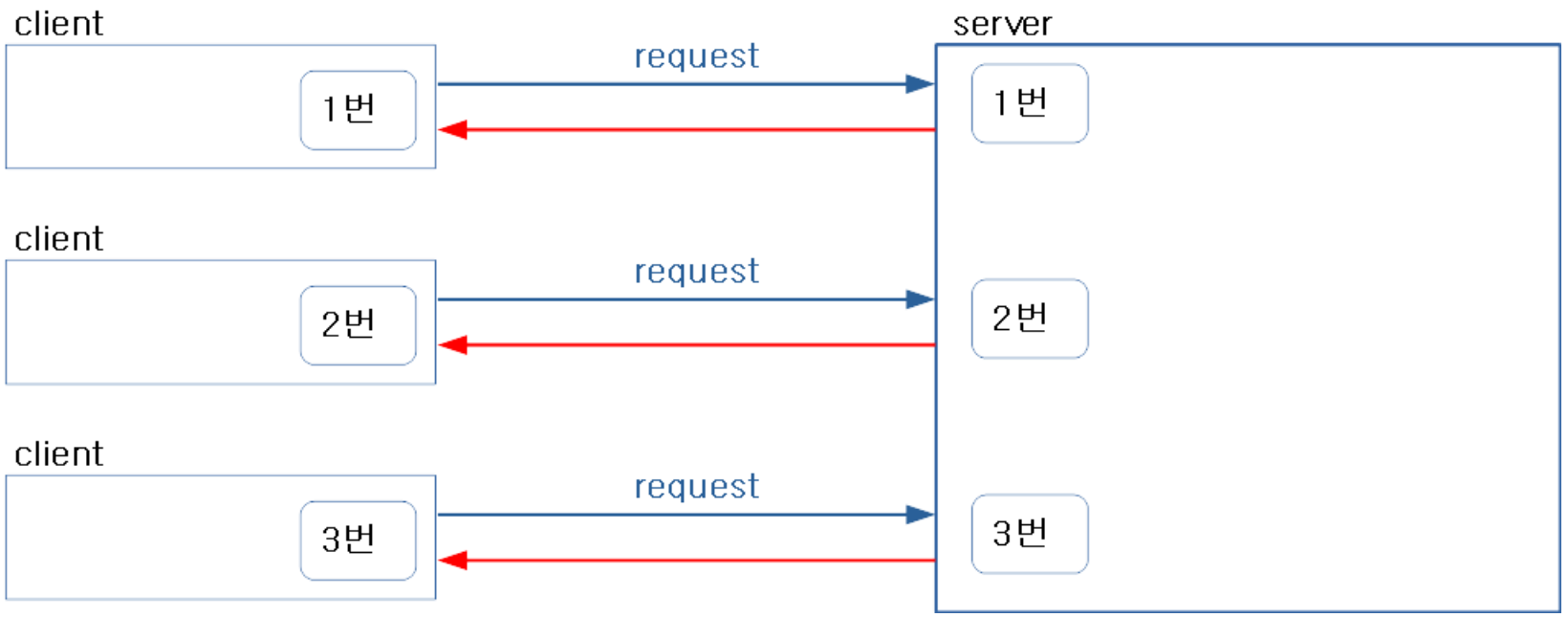
```
@app.route("/get_cookie")
def get_cookie():
  name = request.cookies.get('userID')
  return '<h1>welcome %s</h1>' % name
```

● 실행 결과



■ session

- 클라이언트에 대한 정보를 저장할 수 있는 서버 내의 메모리
- 접속하는 클라이언트 당 하나의 세션 생성
- 사물함과 같은 형식으로 저장되면 사물함의 번호를 클라이언트로 전송
- 클라이언트가 번호를 분실하는 경우 새로운 세션을 생성하여 다시 전송



■ session

● routes.py (세션 저장)

```
from flask import session

@app.route("/save_session/<username>")
def save_session(username):
  session['username'] = username
  return '<h1>Save %s</h1>' % username
```

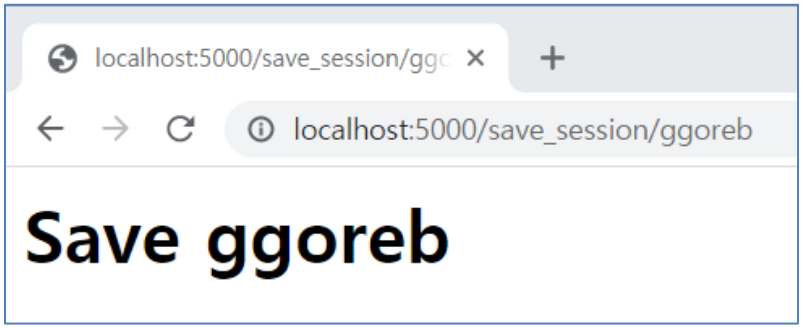
● routes.py (세션 조회)

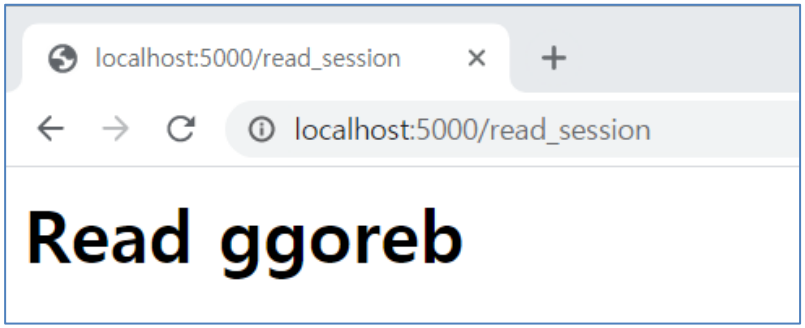
```
@app.route("/read_session")
def read_session():
  username = ''
  try:
    username = session['username']
  except KeyError as e:
    username = 'Unknown'
  return '<h1>Read %s</h1>' % username
```

■ session

● routes.py (세션을 사용하기 위한 설정)

```
from datetime import timedelta

app.secret_key = 'any random string'
app.permanent_session_lifetime = timedelta(minutes=1)
```

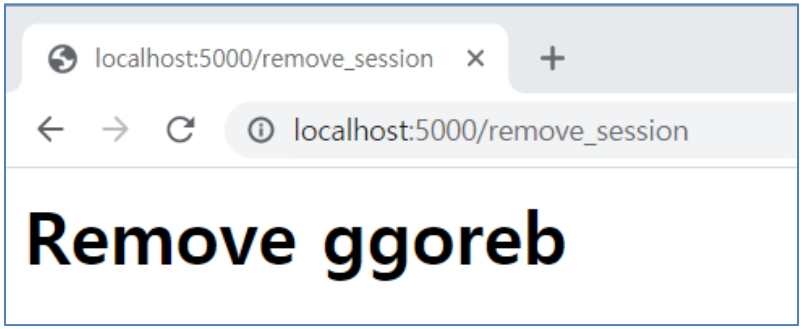


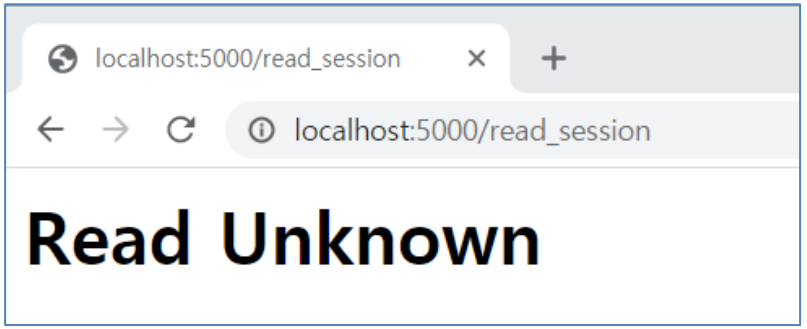


■ session

● routes.py (세션 삭제)

```
@app.route("/remove_session")
def remove_session():
  username = session.pop('username', None)
  return '<h1>Remove %s</h1>' % username
```





■ File upload

- 서버로 파일을 전송하는 방법
- form의 속성 중 method는 post, enctype은 multipart/form-data로 지정

```python
@app.route('/upload', methods=['GET', 'POST'])
def upload():
  if request.method == 'POST':
    f = request.files['file']
    f.save(f.filename)
    return 'File uploaded : %s' % f.filename
  else:
    return render_template('upload.html')
```

- 파일명을 보호하기 위해 secure_filename 함수 사용 가능

```python
from werkzeug.utils import secure_filename
```

사용 전 : 검은사막 OSTCover By FiNE X MUNA (가사).mp3

사용 후 : OSTCover_By_FiNE_X_MUNA_.mp3

■ File download

● 서버에서 클라이언트로 파일을 전송하는 방법

```
from flask import send_file

@app.route('/download')
def download():
    file_name = "OSTCover_By_FiNE_X_MUNA_.mp3"
    return send_file('c:/flask/sample/%s'%file_name,
                     mimetype='text/csv',
                     download_name=file_name,
                     as_attachment=True)
```

■ Database 활용

● 실행순서

1. 데이터베이스 연결
2. Cursor 객체 생성
3. SQL 작성
4. SQL 실행 (조회를 하는 경우 fetchall, fetchmany, fetchone 사용)
5. 변경사항 저장 (commit)
6. Cursor 객체 close
7. 데이터베이스 연결 해제

■ Database 활용

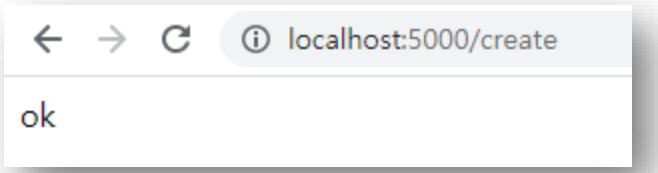
● 데이터베이스 연결 함수 작성

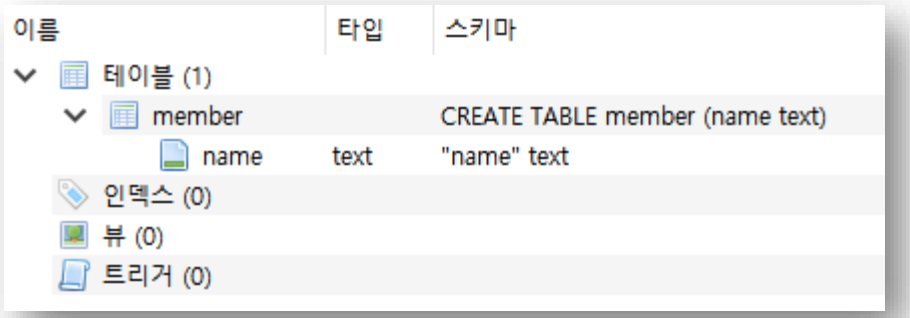
```
import sqlite3

def connect_db():
  return sqlite3.connect('test.db')
```

■ Database 활용

● SQL 작성 및 실행 (Table 생성)

```
@app.route('/create')
def create_db():
  try:
    conn = connect_db()
    sql = 'create table if not exists member (name text)'
    cursor = conn.cursor()
    cursor.execute(sql)
    conn.commit()
    cursor.close()
    conn.close()
  except Exception as e:
    print(e)
  return 'ok'
```





■ Database 활용

● SQL 작성 및 실행 (데이터 조회)

```
from flask import json

@app.route('/select')
def select_db():
  try:
    conn = connect_db()
    sql = 'select * from member'
    cursor = conn.execute(sql)
    result = cursor.fetchall()  데이터 조회
    # result = [ dict(name=r) for r in result ]
    conn.close()
  except Exception as e:
    print(e)
  return json.dumps(result, ensure_ascii=False)  Object → JSON
```